

PullDoctor

Warcraft Logs raid analysis toolkit
Complete user guide & command reference

Guild: Undercover Socials (silvermoon-EU)

Author: Dimitri Crève

Covers the merged module structure: core, wcl, fight_analyzers, boss_mechanics,
consumables, gear, tier_sets, raid_cooldowns, defensive_cooldowns,
damage_targets, reporting, automation

Contents

1. What PullDoctor does
2. Installation & first-time setup
3. Project layout: which file does what
4. Generating reports — `main.py` and `generate_html_report.py`
5. Configuring what gets tracked (the CLI modules)
6. API budget management — `wcl.py` `check-budget`
7. Unattended publishing & GitHub Pages — `automation.py`
8. Reading the HTML report
9. Recommended end-to-end workflows
10. Complete flag reference (all commands)
11. Troubleshooting

1. What PullDoctor does

PullDoctor pulls raid data from the Warcraft Logs v2 GraphQL API, runs a set of analyzers over each pull, and writes three kinds of output: a plain-text summary, a Markdown summary, and a self-contained interactive HTML report you can open in a browser or publish to GitHub Pages.

Analyzer	Answers the question
Damage done / taken / healing	Who did the numbers, and who ate the damage?
Deaths	What killed them, and what damage/healing landed in the 5 s before?
Avoidable damage	Who stood in it, how many times, on which mechanic?
Consumables	Who skipped flask / food / potion / oil / healthstone / Vantus Rune?
Gear	Item level, missing enchants, gem count, gem quality
Tier sets	How many tier pieces (X/5) and at what track quality?
Raid cooldowns	Were Tranquility / Revival / Spirit Link used as often as possible?
Defensive cooldowns	Personal defensive usage, efficiency, and estimated damage prevented
Damage by target	Boss vs adds damage split, with your own target labels

2. Installation & first-time setup

2.1 Requirements

- Python 3.10 or newer (the code uses `X | None` union syntax)
- Packages: `requests`, `python-dotenv`
- A Warcraft Logs API client (free)

```
python --version          # must be 3.10+
pip install requests python-dotenv
```

2.2 Get your API credentials

1. Go to `warcraftlogs.com` → your profile → **API Clients**.
2. Create a new client. Any name and any redirect URL will do (it is not used — PullDoctor uses the `client_credentials` grant).
3. Copy the **Client ID** and **Client Secret**.

2.3 Create your .env file

In the project folder, copy `.env.example` to `.env` and fill it in:

```
WCL_CLIENT_ID=your_client_id_here
WCL_CLIENT_SECRET=your_client_secret_here
```

Never commit `.env`. It is already listed in `.gitignore`. If credentials are missing, every script exits immediately with a message telling you to create the file.

2.4 Set your guild identity

Only needed for the automatic "find our latest log" feature. Open `wcl.py`, Section 3, and edit:

```
GUILD_NAME = "Undercover Socials"
GUILD_SERVER_SLUG = "silvermoon"    # lowercase, exactly as in the WCL URL
GUILD_SERVER_REGION = "eu"         # us | eu | kr | tw | cn
```

Read these straight out of your guild's Warcraft Logs URL:

```
https://www.warcraftlogs.com/guild/eu/silvermoon/Undercover+Socials
           ^^  ^^^^^^^^^^  ^^^^^^^^^^^^^^^^^^^^^^
           region server_slug   guild_name (decode "+" as a space)
```

2.5 Where to find a report code

A Warcraft Logs report URL looks like `https://www.warcraftlogs.com/reports/aBcD1234efGH5678`. The report code is the part after `/reports/` — in this example `aBcD1234efGH5678`. Every command below that takes a `<report_code>` wants exactly that string.

3. Project layout: which file does what

File	Role	Run directly?
main.py	Analyze ONE fight → .txt / .md / optional .html	Yes
generate_html_report.py	Analyze a WHOLE raid night → one .html	Yes
automation.py	Unattended: find latest log, publish to docs/	Yes
wcl.py	API client, credentials, guild identity, budget CLI	Yes (check-budget)
core.py	Data models, log parser, roster, roles, filters, colors	No (library)
fight_analyzers.py	DPS / HPS / damage taken / deaths	No (library)
reporting.py	Text, Markdown, HTML and index rendering	No (library)
consumables.py	Consumable tracking + its own CLI	Yes
gear.py	Gear compliance + its own CLI	Yes
tier_sets.py	Tier piece tracking + its own CLI	Yes
boss_mechanics.py	Avoidable mechanics + its own CLI	Yes
raid_cooldowns.py	Raid CD tracking + its own CLI	Yes
defensive_cooldowns.py	Personal defensives + its own CLI	Yes
damage_targets.py	Target labelling + its own CLI	Yes

Each tracking system stores its configuration in a matching JSON file next to it:

```
consumables.generated.json    gear_requirements.generated.json
tier_sets.generated.json      boss_mechanics.generated.json
raid_cooldowns.generated.json defensive_cooldowns.generated.json
damage_targets.generated.json
```

Every write to these files first saves a .bak copy alongside it, so a bad edit is always recoverable.

4. Generating reports

4.1 main.py — one fight

```
python main.py <report_code> [fight_id] [options]
```

Flag	Type	Default	Meaning
report_code	positional, required	—	The WCL report code
fight_id	positional, optional	prompt	Which pull. Omit to get an interactive picker.
--html	flag	off	ALSO write an HTML file, in addition to .txt/.md
--html-only	flag	off	Write ONLY the HTML file. Implies --html.
--html-output PATH	named	auto	Custom HTML path. .html is forced on regardless.
--include-trash	flag	off	Also allow trash pulls (encounterID == 0)
--min-duration SECS	float	15.0	Skip pulls shorter than this

Examples

```
# 1. Interactive: list the qualifying fights, then pick one
python main.py aBcD1234efGH5678

# 2. Straight to fight #40, text + markdown output
python main.py aBcD1234efGH5678 40

# 3. Same fight, plus an HTML file in reports/
python main.py aBcD1234efGH5678 40 --html

# 4. HTML only - no .txt / .md clutter
python main.py aBcD1234efGH5678 40 --html-only

# 5. HTML to an exact path (extension auto-corrected to .html)
python main.py aBcD1234efGH5678 40 --html --html-output ~/Desktop/ulatek-wipe12

# 6. Review a 9-second early pull that the filter would normally hide
python main.py aBcD1234efGH5678 12 --min-duration 0

# 7. Look at a trash pull (e.g. testing a new consumable rule)
python main.py aBcD1234efGH5678 3 --include-trash --min-duration 0

# 8. Only consider pulls of at least 3 minutes when picking interactively
python main.py aBcD1234efGH5678 --min-duration 180 --html-only
```

Default output of a non-`--html-only` run:

```
./fight_40_report.txt
./fight_40_report.md
./reports/Ulatek-260920262037.html      # only with --html / --html-only
```

4.2 generate_html_report.py — the whole raid night

```
python generate_html_report.py <report_code> [output_path] [options]
```

Flag	Type	Default	Meaning
report_code	positional, required	—	The WCL report code
output_path	positional, optional	auto	Custom file path; .html is guaranteed
--include-trash	flag	off	Include trash pulls in the report
--min-duration SECS	float	15.0	Minimum pull length to include

Examples

```
# 1. Standard: every boss pull of 15s+, auto-named file in reports/  
python generate_html_report.py aBcD1234efGH5678  
  
# 2. Write it somewhere specific  
python generate_html_report.py aBcD1234efGH5678 reports/progress-night-3.html  
  
# 3. Extension is fixed for you - this still produces progress-night-3.html  
python generate_html_report.py aBcD1234efGH5678 reports/progress-night-3  
  
# 4. Serious pulls only (2 minutes+) - cuts noise from instant resets  
python generate_html_report.py aBcD1234efGH5678 --min-duration 120  
  
# 5. Everything, including trash - expensive, use sparingly  
python generate_html_report.py aBcD1234efGH5678 --include-trash --min-duration 0  
  
# 6. Publish straight into the GitHub Pages folder  
python generate_html_report.py aBcD1234efGH5678 docs/reports/HeroicNight.html
```

Auto-naming rule (from reporting.py Section 1):

```
reports/<SanitizedRaidName>-<DDMMYYYY><HHMM>.html  
e.g. reports/TheVenomousAbyss-260920262037.html
```

The timestamp is **when you ran the script**, not when the raid happened. Re-running against the same log produces a second, distinctly named file instead of overwriting the first — handy when you fix a config and want to compare before/after.

5. Configuring what gets tracked

Out of the box, most tracking lists are **empty**. Until you populate them, the gear check passes everyone, tier pieces show 0/5 or n/a, and no avoidable mechanics are flagged. Each system has the same CLI shape:

```
python <module>.py explore # interactive: scan a real log, pick what to track  
python <module>.py list    # show what is currently tracked  
python <module>.py remove  # interactively untrack things  
python <module>.py validate # is the JSON file syntactically valid?  
python <module>.py repair  # auto-fix a trailing-comma JSON error
```

Common `explore` flags across all of these:

Flag	Meaning
<code>--export PATH</code>	Write to a different JSON config file than the default
<code>--replace</code>	Wipe the existing entries on the first successful save this session
<code>--path PATH</code>	(on <code>list</code> / <code>remove</code> / <code>validate</code> / <code>repair</code>) read a different config file

5.1 Consumables — consumables.py

```
python consumables.py explore [report_code] [fight_id] [--export PATH] [--replace]
python consumables.py list [--path PATH]
python consumables.py remove [--path PATH]
python consumables.py remove-category <category> [--yes] [--path PATH]
python consumables.py validate [--path PATH]
python consumables.py repair [--path PATH]
python consumables.py migrate [--yes] [--path PATH]
python consumables.py dump <report_code> <fight_id> [--player NAME]
```

Subcommand	What it is for
<code>explore</code>	Scans a pull, shows every recognised consumable actually used, lets you pick which to track and whether each category is mandatory
<code>remove-category</code>	Drops a whole category (e.g. <code>augment_rune</code>) and all its items
<code>migrate</code>	Applies known one-time fixes: capitalisation renames, enabling loose oil matching, adding missing food entries
<code>dump</code>	Diagnostic: prints raw pre-pull aura names, mid-fight cast names, and weapon temporary-enchant IDs — use this when detection looks wrong

Examples

```
# Build your consumable list from a real pull
python consumables.py explore aBcD1234efGH5678 40

# Start over from scratch
python consumables.py explore aBcD1234efGH5678 40 --replace

# Stop caring about augment runes entirely, no confirmation prompt
python consumables.py remove-category augment_rune --yes

# "Why does it say Tomas has no oil?" - look at the raw data
python consumables.py dump aBcD1234efGH5678 40 --player Tomas

# Apply the known fixes after upgrading
python consumables.py migrate --yes
```

Oil detection: weapon oils are reported as a weapon *temporary enchant*, never as a buff. PullDoctor treats **any** non-empty weapon temporary enchant as "oil used", because the enchant-ID space does not map to spell IDs. If the `dump` output shows an enchant ID, the oil check will pass.

Vantus Rune: detection matches any aura named `Vantus Rune: *`, auto-detected per fight, so it keeps working when the tier changes the suffix.

5.2 Gear compliance — gear.py

```
python gear.py explore [report_code] [fight_id] [--export PATH] [--replace]
python gear.py list [--path PATH]
python gear.py remove [--path PATH]
python gear.py validate [--path PATH]
python gear.py repair [--path PATH]
```

`explore` walks you through, in order:

1. Per-slot enchant rules — for each enchantable slot you can require specific enchant IDs, accept any enchant, or exclude the slot from checking entirely
2. Minimum average item level
3. Minimum quality tier (0=Poor ... 4=Epic ... 5=Legendary)
4. Minimum total gem count
5. A gem-ID whitelist — any socketed gem outside the list fails
6. Free-text profile notes

```
# Build a profile from a night where the roster was mostly correct
python gear.py explore aBcD1234efGH5678 40

# See the active profile
python gear.py list

# Clear one threshold (e.g. the ilvl floor) - interactive picker
python gear.py remove
```

With an empty profile every player passes. That is intentional: a missing rule is never reported as a failure.

5.3 Tier sets — tier_sets.py

```
python tier_sets.py explore <report_code> [fight_id] [--min-players N]
                                     [--include-trash] [--min-duration SECS]
python tier_sets.py list
python tier_sets.py add [--class NAME] [--slot SLOT] [--item-id ID] [--notes TEXT]
python tier_sets.py remove --class NAME --slot SLOT
python tier_sets.py set-breakpoint --track LETTER [--min-ilvl N] [--max-ilvl N]
python tier_sets.py validate
python tier_sets.py repair
```

Tracked slots and their display order: **Head, Shoulder, Chest, Gloves, Legs** (5 total). The report shows a track string such as `MMHC_`:

Letter	Track	Seed item-level range
A	Adventurer	266–282
V	Veteran	279–295
C	Champion	292–308
H	Hero	305–321
M	Myth	318+
	slot not wearing a tracked tier piece	—

```
# Find likely tier items - flags an item_id shared by 2+ same-class players
python tier_sets.py explore aBcD1234efGH5678 40

# Single-player log? Lower the threshold (more false positives)
python tier_sets.py explore aBcD1234efGH5678 40 --min-players 1

# Add a piece non-interactively
python tier_sets.py add --class Paladin --slot Head --item-id 229241 --notes "Season 3"

# Slot can also be given numerically (0=Head 2=Shoulder 4=Chest 9=Gloves 6=Legs)
python tier_sets.py add --class Priest --slot 4 --item-id 229256

# Stop tracking a piece
python tier_sets.py remove --class Paladin --slot Head

# New season shifted the item levels - move the Myth floor to 402
python tier_sets.py set-breakpoint --track M --min-ilvl 402

# Give Hero an explicit ceiling, and clear Myth's ceiling entirely
python tier_sets.py set-breakpoint --track H --min-ilvl 389 --max-ilvl 405
python tier_sets.py set-breakpoint --track M --max-ilvl ""
```

5.4 Avoidable boss mechanics — boss_mechanics.py

```
python boss_mechanics.py explore [report_code] [fight_id] [--export PATH]
                                [--replace] [--show-self-only]
python boss_mechanics.py list [encounter_id] [--path PATH]
python boss_mechanics.py remove <encounter_id> [--path PATH]
python boss_mechanics.py remove-encounter <encounter_id> [--yes] [--path PATH]
python boss_mechanics.py validate [--path PATH]
python boss_mechanics.py repair [--path PATH]
```

`explore` shows two tables:

- **Boss/NPC abilities** — hit counts, unique targets, total/avg/max damage, first-seen time, and whether it caused a death
- **Player-sourced abilities** — carried/soak mechanics, split into *SelfHits* (the carrier, usually fine) and *OtherHits* (someone else got caught, usually the real mistake)

For each ability you select, you are asked:

Prompt	Options	When to use
Track via	damage / debuff / all	Debuff for things that apply a DoT or marker without an immediate hit
Source ≠ target	y / n	Yes for carried mechanics — ignores the carrier's own expected hit
Damage multiplier threshold	e.g. 2	For split/soak damage: only flag hits $\geq 2\times$ the median hit of that pull
Category	free text	ground_effect, cone, spread, split_damage...

```
# Configure mechanics for whichever boss this fight is
python boss_mechanics.py explore aBCd1234efGH5678 40

# Also show player abilities that never hit anyone but their own caster
python boss_mechanics.py explore aBCd1234efGH5678 40 --show-self-only

# Wipe this boss's old config on the first save of the session
python boss_mechanics.py explore aBCd1234efGH5678 40 --replace

# What is configured overall / for one boss?
python boss_mechanics.py list
python boss_mechanics.py list 2917

# Delete an entire boss config without being asked twice
python boss_mechanics.py remove-encounter 2917 --yes
```

5.5 Raid cooldowns — raid_cooldowns.py

```
python raid_cooldowns.py explore <report_code> [fight_id]
                                [--include-trash] [--min-duration SECS]
python raid_cooldowns.py list
python raid_cooldowns.py add [--name NAME] [--cooldown SECS] [--category CAT]
                             [--class NAME] [--notes TEXT]
python raid_cooldowns.py remove --name NAME
python raid_cooldowns.py validate
python raid_cooldowns.py repair
```

```
# Keyword-scan a pull for likely raid cooldowns
python raid_cooldowns.py explore aBcD1234efGH5678 40

# Add one non-interactively
python raid_cooldowns.py add --name "Spirit Link Totem" --cooldown 180 \
    --category healing_cd --class Shaman --notes "Soak phase only"

# Interactive add (prompts for every field)
python raid_cooldowns.py add

# Untrack
python raid_cooldowns.py remove --name "Darkness"
```

`explore` reports an **observed minimum gap** between casts. That is only a lower bound — talents and haste shorten cooldowns, never lengthen them. Always confirm the real base cooldown before adding it.

5.6 Defensive cooldowns — defensive_cooldowns.py

```
python defensive_cooldowns.py explore [report_code] [fight_id] [--export PATH] [--replace]
python defensive_cooldowns.py list [--path PATH]
python defensive_cooldowns.py remove [--path PATH]
python defensive_cooldowns.py validate [--path PATH]
python defensive_cooldowns.py repair [--path PATH]
```

Beyond cast counting, each defensive can carry a **mitigation profile** that drives the "damage prevented" estimate:

Type	Meaning	Estimate produced?
percent_reduction	Flat % reduction for a fixed window (Shield Wall, Barkskin)	Yes — back-calculated from damage actually taken in the window
immunity	Near-total immunity (Ice Block, Divine Shield)	No — shows "n/a" plus any residual damage
unmodeled	Absorb, heal, dodge-chance, redirect, threat drop	No — cast usage only

```
# Add defensives seen in a pull, with mitigation prompts
python defensive_cooldowns.py explore aBcD1234efGH5678 40

# Show the profile of everything tracked, grouped by class
python defensive_cooldowns.py list

# Clean out stale entries for spells removed from the game
python defensive_cooldowns.py remove
```

A "prevented" figure of **n/a** is not a measured zero — it means no estimate is possible for that mitigation type. The HTML report draws these as a hatched bar so they cannot be confused with a real zero.

5.7 Damage-by-target labels — damage_targets.py

```
python damage_targets.py explore [report_code] [fight_id] [--export PATH] [--replace]
python damage_targets.py list [encounter_id] [--path PATH]
python damage_targets.py remove <encounter_id> [--path PATH]
python damage_targets.py remove-encounter <encounter_id> [--yes] [--path PATH]
python damage_targets.py validate [--path PATH]
python damage_targets.py repair [--path PATH]
```

Use this to merge several raw NPC names into one readable label, and to group targets into categories for rollups (boss , miniboss , priority_add , add).

```
# Label the targets seen in a pull
python damage_targets.py explore aBcD1234efGH5678 40

# Review the labels for one boss
python damage_targets.py list 2917

# Start the labelling for that boss again
python damage_targets.py explore aBcD1234efGH5678 40 --replace
```

6. API budget management

The Warcraft Logs v2 API is points-based over a rolling hour. A full raid night can be expensive; check where you stand *before* a big fetch.

```
python wcl.py check-budget
python wcl.py check-budget --json
```

```
# Human-readable
Warcraft Logs API rate-limit status
=====
Points spent this hour:      412.50
Points limit per hour:      3,600.00
Points remaining:           3,187.50 (88.5% left)
Resets in:                   37.2 min
```

```
# Machine-readable, e.g. for a shell script or CI step
python wcl.py check-budget --json | python -c "import json,sys; print(json.load(sys.stdin)
['percent_remaining'])"
```

Warnings are printed automatically below 25 %, more urgently below 10 %, and at 0 % it tells you requests will start failing until the hour rolls over. This command itself uses the cheapest possible query — it fetches no report data at all.

Cheapest way to save budget: leave the default fight filters on. Every trash pull skipped is a full set of Casts / Healing / DamageTaken / DamageDone / Deaths / CombatantInfo / Debuffs queries never made.

7. Unattended publishing & GitHub Pages

7.1 automation.py

```
python automation.py [--recent-days N] [--include-trash]
                    [--min-duration SECS] [--force]
```

Flag	Default	Meaning
<code>--recent-days N</code>	10	Only consider reports uploaded in the last N days
<code>--force</code>	off	Skip the day/time schedule check and run right now
<code>--include-trash</code>	off	Include trash pulls
<code>--min-duration SECS</code>	15.0	Minimum pull length

What it does, in order:

1. Checks the current **Brussels** local time. Unless `--force` is given, it only proceeds on a Monday or Wednesday within 30 minutes of 23:15 — correctly DST-aware via `zoneinfo`, so CET/CEST switchovers cannot misfire it.
2. Verifies the guild identity in `wcl.py` is filled in.
3. Finds your guild's most recent report within the `--recent-days` window. It fetches page 1 *and* the last page and compares timestamps, so it is correct regardless of how the API sorts results.
4. Analyzes every qualifying fight and writes `docs/reports/<RaidName>-<DDMMYYYY><HHMM>.html`.
5. Rebuilds `docs/index.html`, listing every report newest-first, with a meta-refresh straight to the newest one.

```
# Normal scheduled run (what the workflow calls)
python automation.py

# Run it now, regardless of day/time
python automation.py --force

# Guild has not raided in a while - widen the search window
python automation.py --force --recent-days 30

# Long pulls only, forced
python automation.py --force --min-duration 120
```

If it prints "Not running: ... is not one of the scheduled days", that is expected behaviour, not an error. The cron trigger fires more often than the real schedule and this guard is what keeps output to the intended two nights a week. Use `--force` for a manual run.

7.2 Publishing with GitHub Pages

1. Push the project to a GitHub repository.
2. Repository **Settings** → **Pages** → Source: *Deploy from a branch* → branch `main`, folder `/docs`.
3. Add `WCL_CLIENT_ID` and `WCL_CLIENT_SECRET` under **Settings** → **Secrets and variables** → **Actions**.
4. Keep `.github/workflows/scheduled_report.yml` with two cron lines (one for the CET months, one for CEST) — the in-script guard handles the rest.
5. Visit `https://<user>.github.io/<repo>/`; you land on the newest report, with the full history listed underneath.

8. Reading the HTML report

Everything is one self-contained file — no server, no internet needed once generated.

Element	What it tells you
Filter bar (sticky, top)	Filter by Role (tank/healer/melee/ranged), Boss, Difficulty, or type a player name. Reset filters clears everything.
Boss header	Pull count and kill count for that boss
Trend view	Shown for any boss with 2+ pulls: Raid DPS, Raid HPS (tanks/healers only) and Deaths, one bar per pull. Green = kill, red = wipe.
Pull summary line	Difficulty badge, KILL/WIPE badge, pull number, duration, and "Pulled by <player>"
Mini scoreboard	Result, duration, top DPS, top HPS, death count — stays visible while you scroll
Timeline strip	Three lanes across the pull's duration: Deaths (red ticks), Raid CDs (circles), Defensive CDs (squares), class-coloured, hover for detail
Efficiency pills	Red < 40 %, amber 40-70 %, green ≥ 70 %
Meter bars	Class-coloured, scaled to the top row in that table

Section order inside every pull: Damage Done → Healing → Deaths → Avoidable Damage → Damage Taken → Biggest Hits → Defensive Cooldown Usage → Damage Prevented → Raid Cooldown Usage → Gear Check → Consumables. All sections start collapsed; click to expand.

"Pulled by X" is the first player-sourced Casts or DamageDone event in the fight, chronologically. Pet actions fold back to the owner. If no such event exists, the note is omitted rather than guessed.

Biggest Hits excludes tanks when role data is available — a tank eating the biggest hit is their job and would crowd out the interesting entries.

9. Recommended workflows

9.1 First-time setup (about 30 minutes, once)

```
pip install requests python-dotenv
cp .env.example .env          # then fill in your credentials
# edit wcl.py -> GUILD_NAME / GUILD_SERVER_SLUG / GUILD_SERVER_REGION

python wcl.py check-budget      # confirms auth works

# Pick one clean, representative pull and configure everything from it:
python consumables.py          explore <code> <fight>
python gear.py                 explore <code> <fight>
python tier_sets.py            explore <code> <fight>
python defensive_cooldowns.py explore <code> <fight>
python raid_cooldowns.py      explore <code> <fight>

python generate_html_report.py <code>    # sanity-check the result
```

9.2 New boss in progression

```
python boss_mechanics.py explore <code> <fight>    # pick the avoidable mechanics
python damage_targets.py explore <code> <fight>    # label boss vs adds
python boss_mechanics.py list <encounter_id>        # verify
python generate_html_report.py <code>              # regenerate with the new config
```

9.3 After each raid night

```
python wcl.py check-budget
python generate_html_report.py <code> --min-duration 60
# open the newest file in reports/ and review with the raid team
```

9.4 Deep-dive one wipe

```
python main.py <code> <fight_id> --html-only
# or for the text version you can paste into Discord:
python main.py <code> <fight_id>
cat fight_<fight_id>_report.md
```

9.5 New season / new tier

```
python tier_sets.py explore <code> <fight> --min-players 2
python tier_sets.py set-breakpoint --track M --min-ilvl <new floor>
python gear.py explore <code> <fight> --replace          # fresh ilvl/gem profile
python consumables.py explore <code> <fight> --replace    # new flasks/potions
```

10. Complete flag reference

Command	Flags
main.py	report_code, fight_id, --html, --html-only, --html-output, --include-trash, --min-duration
generate_html_report.py	report_code, output_path, --include-trash, --min-duration
automation.py	--recent-days, --include-trash, --min-duration, --force
wcl.py check-budget	--json
consumables.py	explore (--export, --replace), list, remove, remove-category (--yes), validate, repair, migrate (--yes), dump (--player); all support --path
gear.py	explore (--export, --replace), list, remove, validate, repair; all support --path
tier_sets.py	explore (--min-players, --include-trash, --min-duration), list, add (--class, --slot, --item-id, --notes), remove (--class, --slot), set-breakpoint (--track, --min-ilvl, --max-ilvl), validate, repair
boss_mechanics.py	explore (--export, --replace, --show-self-only), list [encounter_id], remove <id>, remove-encounter <id> (--yes), validate, repair; all support --path
raid_cooldowns.py	explore (--include-trash, --min-duration), list, add (--name, --cooldown, --category, --class, --notes), remove --name, validate, repair
defensive_cooldowns.py	explore (--export, --replace), list, remove, validate, repair; all support --path
damage_targets.py	explore (--export, --replace), list [encounter_id], remove <id>, remove-encounter <id> (--yes), validate, repair; all support --path

11. Troubleshooting

Symptom	Cause & fix
"Missing WCL_CLIENT_ID / WCL_CLIENT_SECRET"	No <code>.env</code> in the folder you ran the command from. Create it, or <code>cd</code> into the project directory first.
"Report '...' not found or not accessible"	Wrong code, or the log is private. Copy the code again from the URL after <code>/reports/</code> .
"Fight #N exists in this report, but was filtered out"	It is trash or shorter than the minimum. Add <code>--include-trash</code> and/or <code>--min-duration 0</code> .
"No fights qualify with the current filters"	Same cause at report level — the whole log is trash/short pulls.
Rate-limit errors mid-run	Run <code>python wcl.py check-budget</code> , wait for the reset, and raise <code>--min-duration</code> next time.
"...generated.json is not valid JSON"	Run the module's <code>validate</code> to see the offending line with a caret, then <code>repair</code> — it auto-fixes trailing commas and keeps a <code>.bak</code> .
Everyone shows 0/5 tier pieces	No tier pieces configured yet. Run <code>tier_sets.py explore</code> then <code>add</code> .
Everyone passes the gear check	Empty compliance profile. Run <code>gear.py explore</code> .
No avoidable damage ever reported	No mechanics configured for that encounter ID. Run <code>boss_mechanics.py explore</code> .
A player shows "no gear data"	That pull has no CombatantInfo snapshot for them. This means "unknown", not "failed", and is never counted as a problem.
Oil reported missing for everyone	Run <code>consumables.py dump <code> <fight></code> . If no player has any weapon temporary enchant, the data genuinely is not there for that report.
Vantus Rune says "0 had it" at full uptime	Should no longer happen — detection is prefix-based. Confirm with <code>consumables.py dump</code> ; the detected names are printed in the report too.
Report saved with no extension	Cannot happen — both the auto-named path and any custom path are forced through <code>ensure_html_extension()</code> .
Scheduled job never produces a file	Check the Actions log. "Not running: ..." means the time guard blocked it correctly; verify the cron lines and the guild identity in <code>wcl.py</code> .
A tracked defensive never appears	Some spells were removed or reworked and no longer generate their own cast. Check the entry's notes via <code>defensive_cooldowns.py list</code> .

End of guide. Every configuration write keeps a `.bak` alongside the original, and every report filename is timestamped at generation time, so nothing in this toolkit overwrites work you might still want.